Lab 10 · Friend Classes & Friend Functions in C++

LAB 10

Friend Classes & Friend Functions

Learning Objectives

By the end of this lab, you should be able to:

- Explain why encapsulation sometimes needs a controlled exception — and where friendship fits in.
- Declare a friend class so that one whole class can access another class's private members.
- Declare a friend function so that a single function (member or non-member) can access private members.
- Use forward declarations correctly when two classes need to reference each other.
- Recognize the difference between a friend function and a regular member function.
- Decide when friendship is the right tool — and when it is being misused.

1. Why Do We Need Friendship?

So far in this course we've been told: "keep your data private, expose only what is necessary, control access through public functions." That is encapsulation, and it's a great default.

But sometimes two classes are SO closely related — they were designed together, they're part of the same small subsystem — that forcing one to talk to the other only through public getters and setters becomes awkward and slow. Example: a Library class that has to manage every Book's private quantity field for borrowing and returning. Writing dozens of getters just to support that one collaborator would clutter the Book class for everyone else.

C++ gives us a controlled escape hatch: the friend keyword. By declaring something a friend, a class can grant a specific outside party (another class or a specific function) permission to read and modify its private members directly — while still keeping those members locked away from everyone else.

> 📘 **The mental model — friendship is granted, not taken**
> Friendship is always declared INSIDE the class that owns the private data. A class never "grabs" friendship from outside. It is the data-owner's decision to say: "I trust this particular class (or this particular function) enough to give it the keys to my private fields." Outside code cannot make itself a friend.

Two types of friendship

- *friend class* — grants every member function of an entire class access to all private members. Use when one class is essentially designed to manage another.
- *friend function* — grants access to a single function only. The function can be a free-standing function OR a specific member function of another class. Use when only one specific operation needs privileged access.

Properties of friendship (important rules)

- **Friendship is NOT mutual.** If A declares B as a friend, B can access A's private members. But that does NOT give A access to B's private members.
- **Friendship is NOT inherited.** If A is a friend of B, A's child classes are NOT automatically friends of B.
- **Friendship is NOT transitive.** If A is a friend of B, and B is a friend of C, that does NOT make A a friend of C.

> ⚠ **Use friendship sparingly**
>
> Every friend declaration is a small crack in your encapsulation. If you find yourself sprinkling friend everywhere, you may be designing classes that are too tightly coupled — a sign that the responsibilities should be redistributed or that public methods should be added instead. A good rule of thumb: when in doubt, prefer a public getter; reach for friend only when there is a clear, design-driven reason.

2. Friend Class — Syntax and Setup

Suppose class A wants to grant ALL of class B's member functions access to its private members. Inside A, you write one line: friend class B;

Syntax

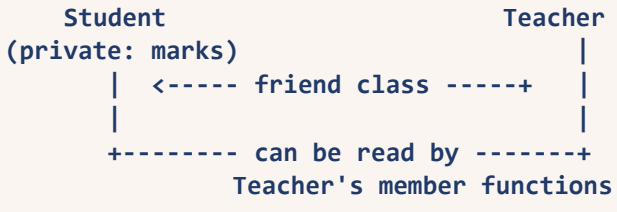
```
class B;                    // forward declaration (often needed)

class A {
private:
    int secret;
    // ... other private stuff
public:
    friend class B;          // <-- B is now a friend of A
};

class B {
public:
    void peekAtA(A obj) {
        cout << obj.secret;  // legal — B is a friend
    }
};
```

> **Why the forward declaration?**
> C++ reads top to bottom. If class A says *friend class B;* before B has been declared, the compiler does not know what B is. The forward declaration *class B;* at the top is our way of telling the compiler: "B is the name of a class — the full definition is coming later." That's enough for the friend statement to be accepted.

3. Code 1 — Teacher as a Friend of Student

Goal: a Student class keeps name, roll number, and marks completely private. A Teacher class needs to calculate the student's grade — which requires reading the private marks. We grant Teacher friendship so it can do its job without us having to expose marks publicly.

> **The friendship relationship**



The Code

```
#include <iostream>
using namespace std;

class Student;                  // forward declaration

class Teacher {
public:
    void calculateGrade(Student s);    // prototype — defined after Student
};

class Student {
private:
    string name;
    int    roll;
    int    marks[3];
public:
    Student(string n, int r, int m1, int m2, int m3) {
        name = n;
        roll = r;
        marks[0] = m1;
        marks[1] = m2;
        marks[2] = m3;
    }
    friend class Teacher;        // <-- Teacher is granted full access
};

void Teacher::calculateGrade(Student s) {
    int total  = s.marks[0] + s.marks[1] + s.marks[2];   // private access!
    double avg = total / 3.0;
    cout << "\nStudent: " << s.name << " (Roll: " << s.roll << ")" << endl;
    cout << "Total Marks: " << total << endl;
    cout << "Average: " << avg << endl;
    if      (avg >= 80) cout << "Grade: A+\n";
    else if (avg >= 70) cout << "Grade: A\n";
    else if (avg >= 60) cout << "Grade: B\n";
    else                cout << "Grade: F\n";
}

int main() {
    Student s1("Rahim", 25, 85, 78, 90);
    Teacher t;
    t.calculateGrade(s1);
    return 0;
}
```

Code Breakdown

Code	What it does
`class Student;`	← Forward declaration — tells the compiler "Student exists" so Teacher can mention it in its prototype below.
`class Teacher { void calculateGrade(Student s); };`	← Teacher is declared first, but calculateGrade is only prototyped — its body cannot be written yet because Student's private fields aren't known.
`class Student { private: ... }`	← name, roll, and marks[3] are all private. Outside code cannot touch them — UNLESS they're a friend.
`friend class Teacher;`	← The magic line. Student grants EVERY member function of Teacher (current and future) access to its private members.
`void Teacher::calculateGrade(Student s)`	← Defined OUTSIDE both classes, after Student is fully known. Now the compiler knows what marks[3] looks like.
`s.marks[0] + s.marks[1] + s.marks[2]`	← Direct access to a private array — legal only because Teacher is a friend. From a non-friend function, this same line would be a compile error.
`t.calculateGrade(s1);`	← Pass the Student object by value. Teacher's member function then peeks into its private data and prints the result.

Expected Output

▶ **Output**

```
Student: Rahim (Roll: 25)
Total Marks: 253
Average: 84.3333
Grade: A+
```

💡 **Notice the design — Student stays clean**

Student doesn't have to add a getMarks(), getName(), or getRoll() function just to support grading. Other parts of the program that don't need to read marks STILL can't read them. Only Teacher gets that privilege — and only because Student explicitly invited Teacher in.

⚠ **Where to put the friend declaration**

Whether you write *friend class Teacher;* under *public:* or *private:* makes no difference — access specifiers do not apply to friend declarations. Most programmers put them at the top or bottom of the class for visibility.

4. Friend Function — Syntax and the Two Flavors

A friend function is a single function that gets access to a class's private members, without itself being a member of that class. It comes in two varieties:

- **A free-standing (non-member) function** — lives outside every class but is granted access via *friend* declaration.
- **A member function of some OTHER class** — for example, *Doctor::checkPatient* could be declared a friend of *Patient* without making the entire Doctor class a friend.

Syntax — both forms

```
class Patient {
private:
    float temperature;
public:
    Patient(float t) { temperature = t; }

    // Form 1: free-standing function as friend
    friend void emergencyAlert(Patient p);

    // Form 2: a SPECIFIC member function of another class as friend
    friend bool Doctor::checkPatient(Patient p);
};

// Definitions live outside, after the class
void emergencyAlert(Patient p) {
    if (p.temperature > 103) cout << "Emergency!";
}

bool Doctor::checkPatient(Patient p) {
    return p.temperature > 100;
}
```

> 📘 **Friend function vs member function**
>
> A **member function** is called on an object: *obj.func()*. It has implicit access to *this*.
>
> A **friend function** is called like an ordinary function: *func(obj)*. The object is passed explicitly. There is no *this* inside it. The *friend* keyword only appears in the DECLARATION inside the class — not when you define the function or when you call it.

5. Code 2 — Doctor, Patient, and Two Friends

Goal: a richer example where one class (Patient) grants two different friendships: (1) a free-standing emergencyAlert function, and (2) one specific member function of the Doctor class. Notice how only the specific function — not all of Doctor — gets the privilege.

The Code

```
#include <iostream>
using namespace std;

class Patient;                   // forward declaration

class Doctor {
public:
    bool checkPatient(Patient p);    // prototype only
    void advice() {
        cout << "Take rest and drink water.\n";
    }
};

class Patient {
    string name;
    int    age;
    float  temperature;
public:
    Patient(string n, int a, float t) {
        name = n;
        age = a;
        temperature = t;
    }
    void showInfo() {
        cout << "Patient Name: " << name << endl;
        cout << "Age: " << age << endl;
    }

    // Two friends, two flavors:
    friend void emergencyAlert(Patient p);              // free-standing
    friend bool Doctor::checkPatient(Patient p);        // a specific member of Doctor
};

bool Doctor::checkPatient(Patient p) {
    cout << "Checking Temperature: " << p.temperature << " C" << endl;
    if (p.temperature > 100) {
        cout << "Diagnosis: Fever detected.\n";
        return true;
    } else {
        cout << "Diagnosis: Normal.\n";
        return false;
    }
}

void emergencyAlert(Patient p) {
    if (p.temperature > 103)
        cout << "Emergency Alert! Immediate hospitalization needed.\n";
    else
        cout << "No emergency.\n";
}

int main() {
    Patient p1("Rahim", 22, 104.5);
```

```
    Doctor  d;
    p1.showInfo();
    if (d.checkPatient(p1)) {
        d.advice();
        emergencyAlert(p1);
    }

    Patient p2("Karim", 23, 99);
    p2.showInfo();
    if (d.checkPatient(p2)) {
        d.advice();
        emergencyAlert(p2);
    }
    return 0;
}
```

Code Breakdown

Code	What it does
`class Patient;`	← Forward declaration — Doctor's checkPatient prototype mentions Patient by name.
`bool checkPatient(Patient p);`	← Just a prototype inside Doctor. The body comes after Patient is fully defined.
`void advice() { ... }`	← An ordinary Doctor member function — NOT a friend of Patient. It doesn't need access to Patient's private data, so no friendship needed.
`friend void emergencyAlert(Patient p);`	← Form 1: a free-standing function that will be granted access to Patient's private temperature.
`friend bool Doctor::checkPatient(Patient p);`	← Form 2: ONLY this one Doctor function becomes a friend. Doctor::advice still cannot touch Patient's private members.
`p.temperature`	← Direct access to a private field — works inside both checkPatient AND emergencyAlert, because both are friends.
`d.checkPatient(p1)`	← Called like a normal member function on a Doctor object — the friend keyword is invisible at the call site.
`emergencyAlert(p1)`	← Called like a free function — no object on the left. Patient is passed as an argument.

Expected Output

▶ **Output**

```
Patient Name: Rahim
```

```
Age: 22
Checking Temperature: 104.5 C
Diagnosis: Fever detected.
Take rest and drink water.
Emergency Alert! Immediate hospitalization needed.
Patient Name: Karim
Age: 23
Checking Temperature: 99 C
Diagnosis: Normal.
```

Why grant friendship to ONE function instead of the whole class?

Look closely: *advice()* is a member of Doctor, but it is NOT a friend of Patient. That's intentional — *advice* doesn't need to read Patient's private data, so it shouldn't have permission to. By granting friendship to *checkPatient* alone (instead of writing *friend class Doctor*), we keep the principle of least privilege intact.

Trace what runs for Karim (the second patient)

Karim's temperature is 99 — not above 100 — so *checkPatient* prints "Diagnosis: Normal" and returns false. Because the *if* in main fails, the advice and emergency-alert calls for Karim are SKIPPED. That's why his block of output is two lines shorter than Rahim's.

6. Quick Reference

Friend class vs friend function

Aspect	friend class B	friend function f()
What gets access	Every member function of B	Just that one function f
Declaration syntax	friend class B;	friend return-type f(args);
Best for	Two classes designed together as a tight pair	One specific operation that needs privileged access
Risk to encapsulation	Higher — exposes all of B	Lower — minimal surface area

Friendship — what it is and isn't

Property	Friendship behavior
Granted by	The class with the private data (the giver)
Direction	One-way (A → B does NOT mean B → A)
Inheritance	NOT inherited — child classes are not automatic friends
Transitivity	NOT transitive — friend-of-friend is not a friend
Affected by access specifiers (public/private)?	No — friend declarations ignore them

7. Summary

Five things to take away from this lab:

- **Friendship is a controlled exception to encapsulation.** Use it to grant ONE class or ONE function access to private data — never as a shortcut for not designing your interface.
- **friend class X grants access to ALL of X's member functions.** Use it when X is the natural manager of the data (e.g., Library managing Books).
- **friend function lets you grant access to a SINGLE function.** Use it when only one specific operation needs privileged access — much safer than granting the whole class.
- **Forward declarations are often required** when two classes refer to each other in their declarations.
- **Friendship is one-way, not inherited, not transitive.** Be aware of these limits — they prevent friendship from accidentally spreading.

8. Practice Tasks

Task 1 — Bank and Account (friend class)

Build a small banking system where the Bank class needs deep access to Account objects to perform transactions.

Class Account

- Private data members: holderName (string), accountNumber (int), balance (double).
- Constructor that initializes all three.
- Public showInfo() that displays holder name and account number ONLY (not balance — that's private business).
- Declare class Bank as a friend so the Bank can read and modify the balance directly.

Class Bank

- A public deposit(Account &acc, double amount) function that adds amount to the account's balance and prints a confirmation.
- A public withdraw(Account &acc, double amount) function that subtracts the amount if the balance is sufficient. If not, print "Insufficient balance!" and don't change anything.
- A public showBalance(Account acc) function that prints the holder's name and current balance.

In main()

- Create one Account and one Bank object.
- Call showInfo() on the account, then have the Bank perform: a successful deposit, a successful withdrawal, and a failed withdrawal.
- Call showBalance() after each transaction to verify the changes.

> **Hint for Task 1**
> Pass Account by REFERENCE (*Account &acc*) to *deposit* and *withdraw* so changes actually stick. Passing by value would only modify a COPY, and you'd be left wondering why the balance never updates. Use *Bank::* functions to access *acc.balance* directly — that's the whole point of friendship.

Task 2 — Box Class with a friend Function (combineBoxes)

Build a Box class that represents a 3D box, and a free-standing friend function that can combine two boxes into a third.

Class Box

- Private data members: length, width, height (all double).
- Constructor that initializes the three dimensions.
- Public displayBox() that prints the dimensions in a clean format like "Box: 3 x 4 x 5".
- Public volume() function that returns length * width * height.

Friend function (NOT a member of Box)

- Declare a friend function: *Box combineBoxes(Box b1, Box b2);*
- Define the function OUTSIDE the class. It should create and return a new Box whose length is the SUM of the two input lengths (same for width and height).
- The function must directly access the private length, width, height of both input boxes — which is only legal because it is a friend.

In main()

- Create two Box objects with different dimensions.
- Display both, then call combineBoxes to make a third one and display that too.
- Print the volume of all three using volume().

> 📘 **What you should observe**
>
> Create *Box b1(2, 3, 4); Box b2(1, 2, 3);*. After *Box b3 = combineBoxes(b1, b2);*, b3 should be a 3 × 5 × 7 box with volume 105. If your output matches that, your friend function is correctly accessing both boxes' private fields.

> ⚠ **Common mistake to avoid**
>
> Do NOT write *friend* when defining the function — only when DECLARING it inside the class. The definition outside looks like an ordinary function: *Box combineBoxes(Box b1, Box b2) { ... }*. Writing *friend* in the definition is a compile error.

Prepared by

Alifa Shanzidah Manarat

Lecturer

Dept. of CSE, BAUST